Individual Project Report

Chaitanya Nikhil Vikamsey

LC5: Team 10

ENGR 133: Transforming Ideas to Innovation

Prof. Ben Manning

12/01/2025

1. Project Introduction

My Individual Project is called "Manufacturing Made Easier", which is a program that takes constraints from the user and predicts (using two interconnected Machine Learning Algorithms) the most suitable material, the most suitable Machining Process to manufacture a specific part, and the cost it would take to manufacture said part. This program makes the manufacturing process easier for the user, since they have effectively bypassed the first three steps in the process of manufacturing a part (choice of materials, choice of machining process, and prediction of cost), saving them a lot of invaluable time. My dream project was to be able to make machines that can make other machines (like a 3d printer that can print another 3d printer and so on), and for this goal to be met, the process of manufacturing must be automated; therefore, I started by automating the first three steps. These steps usually take a long time in organizations, since they must do extensive research, run trade studies, talk to experienced people, and calculate costs to be able to determine the final chosen material and machining process to manufacture the part. This program effectively saves them all that time, since it automatically gives them the answers that they need. It is also pivotal in my dream project because if the machine doesn't know what material it's going to use and the process needed to create the part, it cannot be manufactured. Therefore, my program is pivotal to automating and speeding up the process of manufacturing.

2. Project Overview of Inputs and Outputs

This project is a manufacturing decision-support tool designed to help users identify the most suitable material and manufacturing process for a given part based on engineering requirements. The program accepts several user-defined inputs describing the part's geometry and performance needs. Using these inputs, the system evaluates a database of materials and processes, applies a trained machine-learning model with heuristic checks, and outputs ranked recommendations along with a full cost breakdown.

Inputs

1. **Part Volume (cm^3)** – Used to estimate material usage and machining/printing cycle time.
2. **Surface Area (cm^2)** – Influences process selection and processing time.
3. **Tolerance (mm)** – Determines which manufacturing processes can achieve the required precision.
4. **Quantity (number of parts)** – Used only for cost calculation (not model training).
5. **Minimum Yield Strength (MPa)** – Filters materials that do not meet structural requirements.
6. **Maximum Density (g/cm^3)** – Removes overly heavy materials.
7. **Maximum Cost ($/kg)** – Ensures material options fit user budget limitations.

Outputs

- **Top Recommended Material** based on a combined ML–heuristic score.
- **Top Recommended Manufacturing Process** with associated probability scores.
- **Full Cost Breakdown File**, including material cost, processing cost, and total cost.
- **Bar Chart of Process Probabilities** saved to the output folder.

Example

Input:

Volume = 120 cm^3, Surface Area = 350 cm^2, Tolerance = 0.1 mm, Quantity = 10,
Min Strength = 200 MPa, Max Density = 3.0 g/cm^3, Max Cost = $12/kg

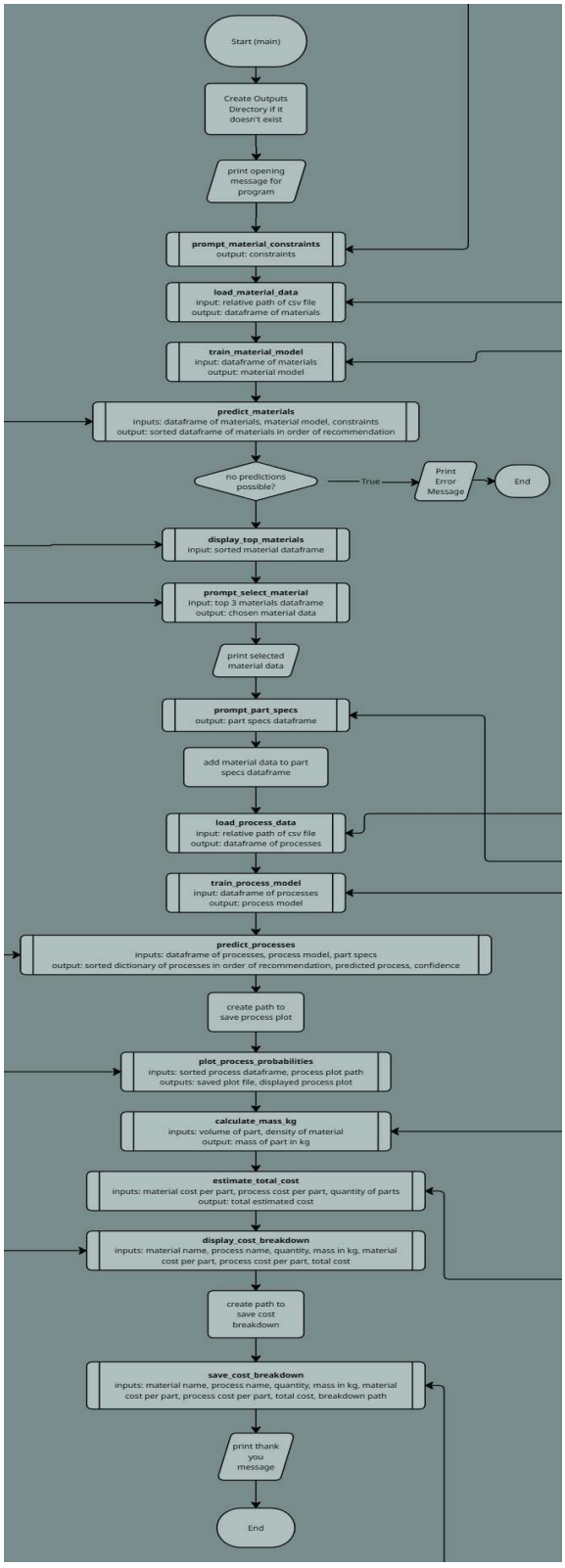
Output:

- Material: *6061-T6 Aluminum* (score 87.3)
- Process: *CNC Machining* (confidence 0.74)
- Total Cost: *$46.80* for 10 parts
- Cost breakdown report saved as a text file, Probability plot saved as PNG

3. User-defined Functions

1. Main

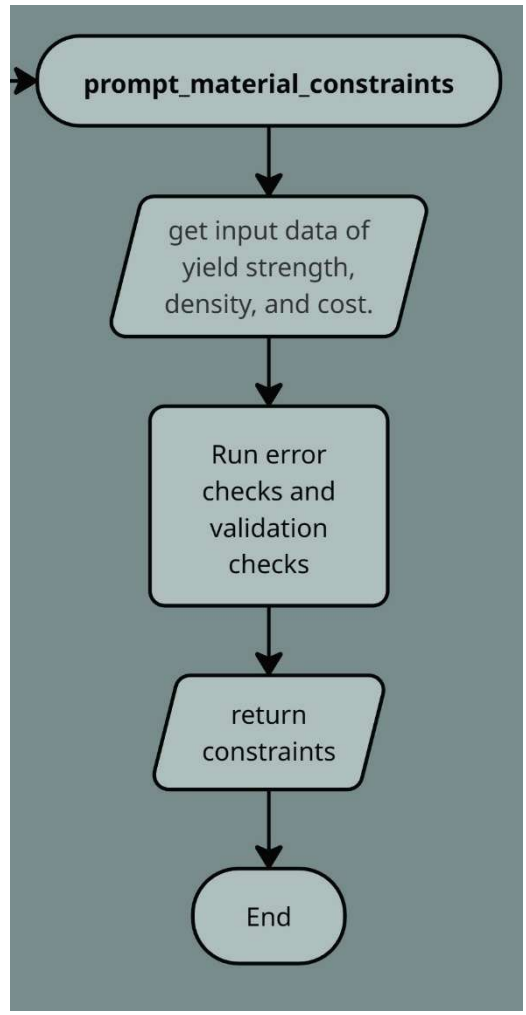
```python
def main():
    OUTPUT_DIR = "output"
    os.makedirs(OUTPUT_DIR, exist_ok=True)                #Creating output directory if it doesn't exist, so that I can store the process probability plots there
    print("====== Manufacturing Made Easier (Material, Process, and Cost Estimator) ======")
    #collecting user constraints for material selection
    constraints = prompt_material_constraints()
    #loading material data and training material selection model
    mat_df = load_material_data("data/materials.csv")
    mat_model = train_material_model(mat_df)
    #predicting materials based on user constraints and displaying the top recommendations
    mat_predictions = predict_materials(mat_model, mat_df, constraints)
    if mat_predictions.empty:
        print("\nNo materials found even after relaxation. Please try again with different constraints.")   #handling case where no materials are found even after constraint relaxation
    else:
        display_top_materials(mat_predictions)
        #prompting user to select one of the top 3 materials and then printing the selected material and its properties
        top3 = mat_predictions.head(3)
        selected_material = prompt_select_material(top3)
        print("\nSelected material:")
        print(selected_material.to_string())
        #collecting part specifications from user and adding material properties to it
        part_specs = prompt_part_specs()
        part_specs["material_name"] = selected_material["name"]
        part_specs["material_density_g_cm3"] = float(selected_material["density_g_cm3"])
        part_specs["material_strength_MPa"] = float(selected_material["yield_strength_MPa"])
        part_specs["material_cost_usd_per_kg"] = float(selected_material["cost_usd_per_kg"])
        #loading process data, training process model, predicting processes for the part, and plotting the process probabilities
        proc_df = load_process_data("data/process_train.csv")
        proc_model = train_process_model(proc_df)
        proc_result = predict_processes(proc_model, part_specs)
        proc_plot_path = os.path.join(OUTPUT_DIR, "process_probabilities.png")
        plot_process_probabilities(proc_result, proc_plot_path)
        #calculating mass, costs, and displaying & saving cost breakdown
        mass_kg = calculate_mass_kg(part_specs["volume_cm3"], part_specs["material_density_g_cm3"])
        process_name = proc_result["predicted_process"]
        material_cost_per_part = part_specs["material_cost_usd_per_kg"] * mass_kg
        process_cost_per_part = PROCESS_COST_MAP.get(process_name, PROCESS_COST_MAP["Other"])
        total_cost = estimate_total_cost(material_cost_per_part, process_cost_per_part, part_specs["quantity"])
        display_cost_breakdown(
            selected_material["name"],
            process_name,
            part_specs["quantity"],
            mass_kg,
            material_cost_per_part,
            process_cost_per_part,
            total_cost
        )
        out_report_path = os.path.join(OUTPUT_DIR, "cost_breakdown_report.txt")
        save_cost_breakdown(
            selected_material["name"],
            process_name,
            part_specs["quantity"],
            mass_kg,
            material_cost_per_part,
            process_cost_per_part,
            total_cost,
            out_report_path
        )
        #ends with a thank you message
        print("\nProcess probability chart and Cost Breakdown Report saved to output folder.")
        print("All done. Thank you for using Manufacturing Made Easier!")
```



The main function ties together all the UDFs and arranges them in a way that can allow us to reach the final logic that the program has to serve.

2. Prompt_material_constraints

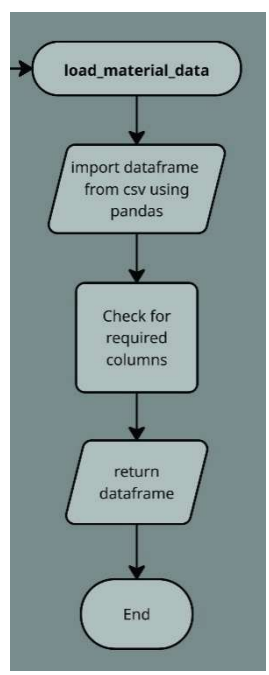
```python
def prompt_material_constraints():
    while True:
        try:                                                    #User input with validation, using try except blocks
            strength = float(input("Enter minimum required yield strength (MPa): "))
            if strength <= 0:
                print("Strength must be greater than 0.")
                continue
            break
        except ValueError:
            print("Invalid input. Please enter a numeric value.")

    while True:
        try:
            density = float(input("Enter max allowable density (g/cm^3): "))
            if density <= 0:
                print("Density must be greater than 0.")
                continue
            break
        except ValueError:
            print("Invalid input. Please enter a numeric value.")

    while True:
        try:
            cost = float(input("Enter maximum cost ($/kg): "))
            if cost <= 0:
                print("Cost must be greater than 0.")
                continue
            break
        except ValueError:
            print("Invalid input. Please enter a numeric value.")

    return {                            #Returning constraints as a dictionary
        "target_strength": strength,
        "max_density": density,
        "max_cost": cost
    }
```



This function prompts the user to enter the constraints for the material choice, so that the ML model can go ahead and run it to get the recommended materials. It also has try-except blocks that validate the inputs and make sure they are usable.

3. Load_material_data

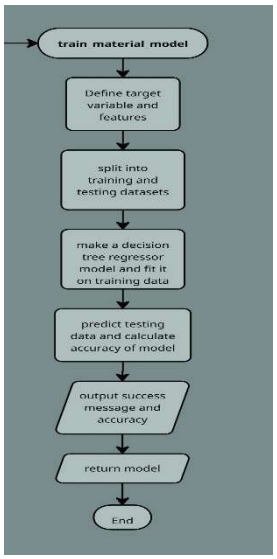
```python
def load_material_data(path):
    df = pd.read_csv(path)
    required = ["name", "density_g_cm3", "yield_strength_MPa", "cost_usd_per_kg",
                "corrosion_rating", "success_score"]
    for col in required:                                    #Checking for required columns in the CSV
        if col not in df.columns:
            raise ValueError(f"Missing column '{col}' in materials CSV.")
    return df.copy()                                #Returning a copy of the dataframe
```



This code loads the material data from the csv file that is used to train the model.

4. Train_material_model

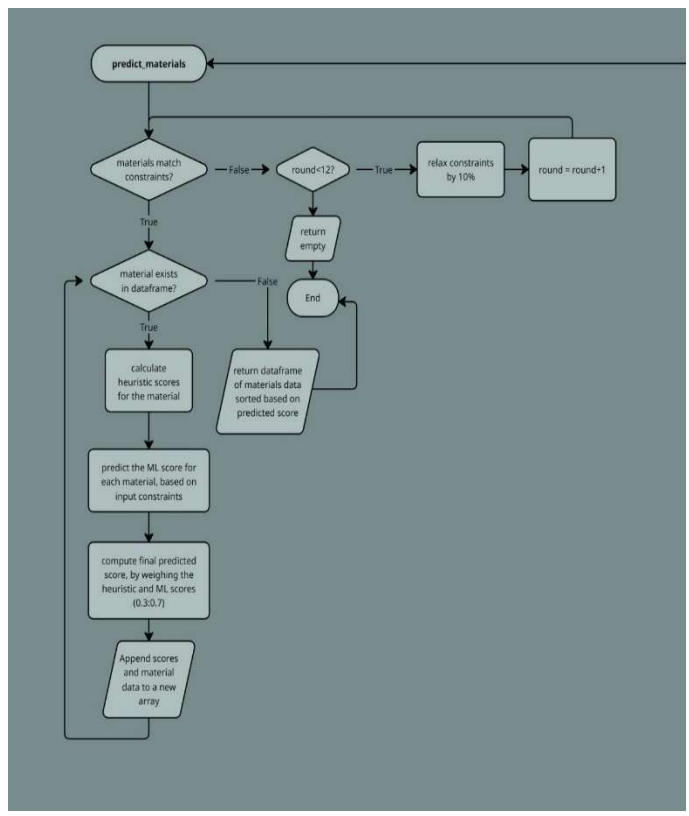
```python
def train_material_model(df):
    features = df[["density_g_cm3", "yield_strength_MPa", "cost_usd_per_kg",
                   "corrosion_rating"]].astype(float)                    #Defining features and target variable for model training
    target = df["success_score"].astype(float)

    X_train, X_test, y_train, y_test = train_test_split(features, target, test_size=0.2, random_state=42)      #splits data into train and test
    model = DecisionTreeRegressor(max_depth=5, random_state=42)                #Creating a decision tree regressor model
    model.fit(X_train, y_train)                        #Training the decision tree regressor model on training data

    preds = model.predict(X_test)                  #Making predictions on test data
    metrics = {
        "r2": float(r2_score(y_test, preds)),
        "mae": float(mean_absolute_error(y_test, preds))
    }
    print(f"Material model trained. \nR2={metrics['r2']:.3f}(Higher = Better), \nMAE={metrics['mae']:.3f}(Lower = Better)")     #Printing model performance metrics, for user to know accuracy of Model
    return model
```



This function trains a decision tree regressor on the training set of data and then validates it using the test set of data and displays the accuracy of the model while returning the model to the main code.

5. Predict_materials

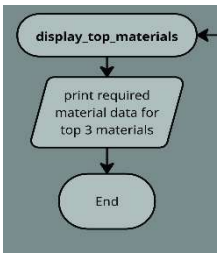
```python
def predict_materials(model, df, constraints):
    relaxation_round = 0
    print("\nSearching for materials that meet your constraints... \nConstraints will be relaxed by 10% per round of relaxing the constraints, if no materials meet your constraints.")
    while True:
        #only materials meeting constraints remain
        df = df[
            (df["yield_strength_MPa"] >= constraints["target_strength"]) &
            (df["density_g_cm3"] <= constraints["max_density"]) &
            (df["cost_usd_per_kg"] <= constraints["max_cost"])
            ]

        #If at least one material matches, don't relax constraints
        if not df.empty:
            break

        #If no materials found, then relax constraints

        # asking the user to reconsider their constraints if too many relaxations have occurred
        if relaxation_round > 12:   # roughly 12 relax cycles, should be enough, and if it isnt, the constraints are too tight
            return pd.DataFrame([])  # return empty datafame to indicate no filtering possible, and constraints are too tight
        else:
            relaxation_round += 1
            print(f"Relaxing Constraints -- Round {relaxation_round}...")
            #Relaxation rules
            constraints["target_strength"] *= 0.9       # reduce strength by 10%
            constraints["max_density"] *= 1.1           # increase density by 10%
            constraints["max_cost"] *= 1.1              # increase cost by 10%

    if 12 > relaxation_round > 0:     # Inform user of final relaxed values, if it happens within 12 cycles
            print("\n Constraints relaxed to allow valid materials:")
            print(f"  Final target strength: {constraints["target_strength"]:.2f} MPa")
            print(f"  Final max density:     {constraints["max_density"]:.2f} g/cm³")
            print(f"  Final max cost:        $({constraints["max_cost"]:.2f}/kg")

    # Scoring remaining materials with the trained model
    rows = []
    for _, row in df.iterrows():
        # Calculating heuristic scores based on constraints for qualitative aspect of analysis and material recommendation
        # Strength score
        s_strength = min(row["yield_strength_MPa"] / max(1.0, constraints.get("target_strength", 1.0)), 1.0)
        # Cost score
        s_cost = 1 - min(row["cost_usd_per_kg"] / max(1.0, constraints.get("max_cost", row["cost_usd_per_kg"])), 1.0)
        # Density score
        dens_min = df["density_g_cm3"].min()
        dens_max = df["density_g_cm3"].max()
        if dens_max - dens_min > 0:
            s_density = 1 - (row["density_g_cm3"] - dens_min) / (dens_max - dens_min)
        else:
            s_density = 0.5

        # Predicting quantitative score using the trained decision tree regressor model
        X = pd.DataFrame([{
            "density_g_cm3": row["density_g_cm3"],
            "yield_strength_MPa": row["yield_strength_MPa"],
            "cost_usd_per_kg": row["cost_usd_per_kg"],
            "corrosion_rating": row["corrosion_rating"]
        }])
        predicted = model.predict(X)[0]

        # Combining quantitative and qualitative scores for final material recommendation
        heuristic = ((s_strength + s_cost + s_density) / 3.0)*100
        combined_score = 0.7 * predicted + 0.3 * (heuristic)

        rows.append({
            "name": row["name"],
            "density_g_cm3": row["density_g_cm3"],
            "yield_strength_MPa": row["yield_strength_MPa"],
            "cost_usd_per_kg": row["cost_usd_per_kg"],
            "corrosion_rating": row["corrosion_rating"],
            "predicted_score": combined_score,
            "quantitative_analysis_score": predicted,
            "qualitative_analysis_score": heuristic
        })

    out = pd.DataFrame(rows)
    out = out.sort_values("predicted_score", ascending=False).reset_index(drop=True) #Sorting materials by predicted score in descending order
    return out
```



This function predicts the most appropriate materials for the constraints that the user has provided, using heuristics and machine learning in tandem with each other to give a holistic recommendation of materials. It also checks for realistic constraints and otherwise relaxes to give a near perfect recommendation if not perfect.

6. Display_top_materials

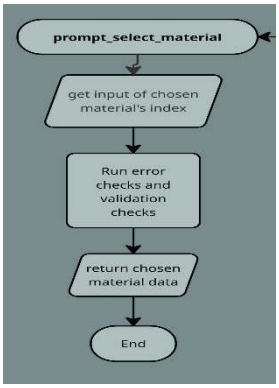
```python
def display_top_materials(df, n=3):
    print("\nTop recommended materials:")        #Displaying top N recommended materials, keeping n as 3 by default, but changeable as needed
    for i in range(min(n, len(df))):
        r = df.iloc[i]
        print(f"{i+1}. {r['name']} | Score: {r['predicted_score']:.1f} | Cost: ${r['cost_usd_per_kg']}/kg | Density: {r['density_g_cm3']} g/cm3")
```



This function displays the top recommended materials for the user to see and to choose from.

7. Prompt_select_material

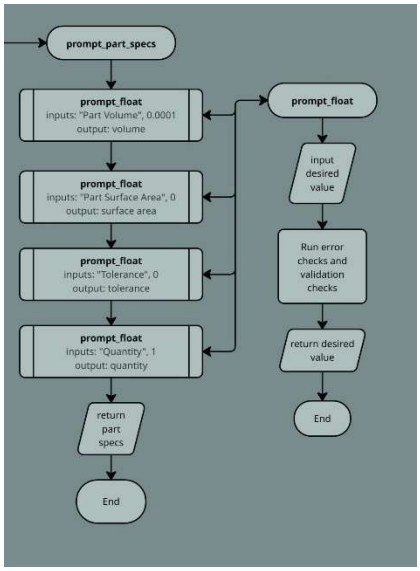
```python
def prompt_select_material(top3_df):
    while True:                                  #Prompting user to select one of the top 3 recommended materials, using try except and an infinite loop until we get a valid value.
        try:
            choice = input("Select material to use (1, 2 or 3): ").strip()
            if choice not in ("1", "2", "3"):
                print("Please enter 1, 2, or 3.")
                continue
            id = int(choice) - 1
            selected = top3_df.iloc[id]
            return selected
        except Exception as e:
            print("Invalid choice:", e)
```



This Function prompts the user to select one of the top recommended materials to go ahead with the manufacturing process.

8. Prompt_part_specs

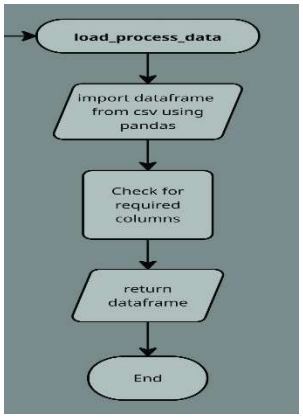
```python
def prompt_part_specs():
    def prompt_float(name, min_val=None):                    #using a nested function to prompt for float inputs with validation so that it is easier
        while True:                                     #to call as long as a valid input is given
            try:
                s = input(f"\nEnter {name}: ").strip()
                val = float(s)
                if min_val is not None and val < min_val:
                    print(f"{name} must be >= {min_val}. Try again.")
                    continue
                return val
            except ValueError:
                print("Please enter a numeric value.")

    volume = prompt_float("Part volume (cm^3)", min_val=0.0001)                #Prompting user for part specifications using previoulsy defined UDF
    surface_area = prompt_float("Surface area (cm^2)", min_val=0.0)
    tolerance = prompt_float("Tolerance (mm)", min_val=0.0)
    qty = int(prompt_float("Quantity (integer)", min_val=1))
    return {
        "volume_cm3": volume,
        "surface_area_cm2": surface_area,
        "tolerance_mm": tolerance,
        "quantity": qty
    }
```



This function prompts the user to enter the specifications of the part that they want to manufacture.

9. Load_process_data

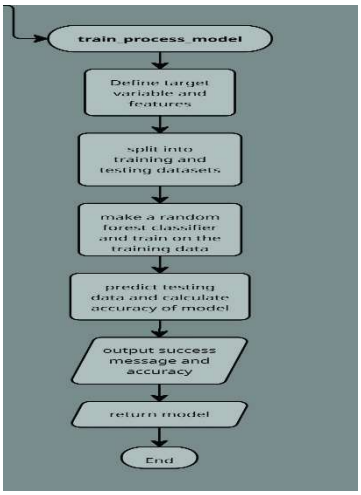
```python
def load_process_data(path):
    df = pd.read_csv(path)
    required = ["volume_cm3", "surface_area_cm2", "tolerance_mm",
                "material_strength", "material_density", "process_label"]
    for col in required:                                                    #Validating presence of required columns in CSV
        if col not in df.columns:
            raise ValueError(f"Missing column '{col}' in process CSV.")
    return df.copy()
```



This Function loads the process data from the csv for the machine learning model to be trained on.

10. Train_process_model

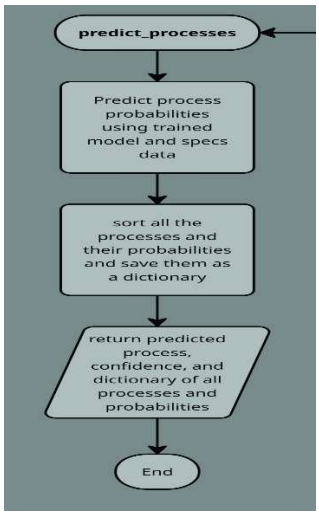
```python
def train_process_model(df):
    X = df[["volume_cm3", "surface_area_cm2", "tolerance_mm",
            "material_strength", "material_density"]].astype(float)        #defining feature matrix and target vector
    y = df["process_label"].astype(str)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
    model = RandomForestClassifier(n_estimators=150, random_state=42)                #Using Random Forest Classifier with 150 trees for better accuracy
    model.fit(X_train, y_train)                                                 #Training the model, and then evaluating on test set
    preds = model.predict(X_test)
    acc = accuracy_score(y_test, preds)
    print(f"Process model trained. Test accuracy: {acc:.3f}")            #Printing model accuracy on test set for the user to know
```



This function trains a random forest classifier on the training set of process data and then tests it on the test set of data, prints the accuracy and returns the model.

11. Predict_processes

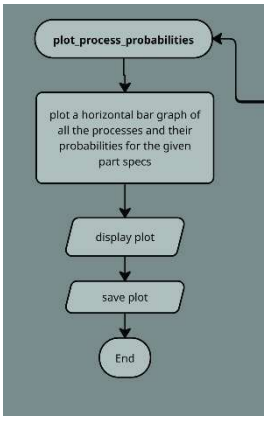
```python
def predict_processes(model, specs):
    X = pd.DataFrame([{                                                 #Creating a dataframe from user input specifications for prediction
        "volume_cm3": specs["volume_cm3"],
        "surface_area_cm2": specs["surface_area_cm2"],
        "tolerance_mm": specs["tolerance_mm"],
        "material_strength": specs["material_strength_MPa"],
        "material_density": specs["material_density_g_cm3"],
    }])
    proba = model.predict_proba(X)[0]                                   #Predicting process probabilities using the trained model
    classes = model.classes_                                            #Getting the class labels
    pairs = list(zip(classes, proba))                                   #Zipping class labels with their corresponding probabilities
    pairs_sorted = sorted(pairs, key=lambda x: x[1], reverse=True)      #Sorting the pairs based on probabilities in descending order
    predicted_process, confidence = pairs_sorted[0]                     #Getting the process with highest probability and its confidence
    all_probs = {name: float(prob) for name, prob in pairs_sorted}      #Creating a dictionary of all process probabilities
    print(f"\nPredicted process: {predicted_process} (confidence {confidence:.3f})")
    return {"predicted_process": predicted_process, "confidence": float(confidence), "all_probs": all_probs}
```



This function predicts the most suitable process based on the specifications given by the user and the material chosen by the user.

12. Plot_process_probabilities

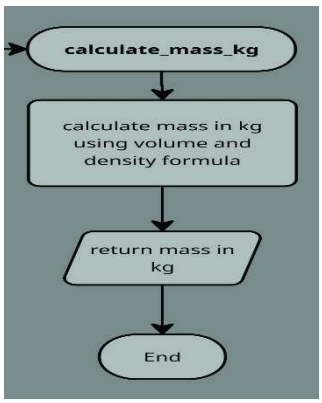
```python
def plot_process_probabilities(proc_result, out_path):
    probs = proc_result["all_probs"]                          #Plotting the process prediction probabilities as a horizontal bar chart
    names = list(probs.keys())
    vals = [probs[n] for n in names]

    plt.figure(figsize=(10, 10))
    plt.barh(names, vals, color="C8")
    plt.xlabel("Probability")
    plt.title("Process prediction probabilities")
    plt.xlim(0, 1)
    for i, v in enumerate(vals):
        plt.text(v + 0.01, i, f"{v:.2f}", va="center")          #Adding text labels to each bar indicating the probability value
    plt.tight_layout()
    plt.savefig(out_path, dpi=150)                                  #Saving the plot to the specified output path
    plt.show()
    plt.close()
```



This function plots the process probabilities in a horizontal bar chart for the user to see the probabilities of all the processes, it also saves the plot for future reference.

13. Calculate_mass_kg

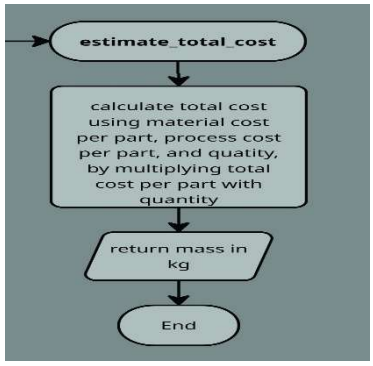
```python
def calculate_mass_kg(volume_cm3, density_g_cm3):
    mass = (float(volume_cm3) * float(density_g_cm3)) / 1000.0        #Calculating mass in kg from volume in cm^3 and density in g/cm^3
    return mass
```



This function uses the volume and the density to calculate the mass of the part manufactured in kilograms.

14. Estimate_total_cost

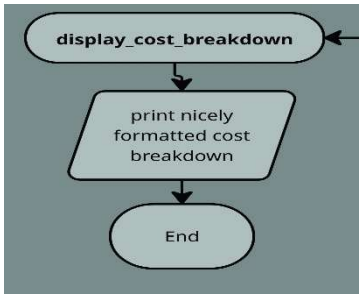
```python
def estimate_total_cost(material_cost_per_part, process_cost_per_part, quantity):
    total = (float(material_cost_per_part) + float(process_cost_per_part)) * int(quantity)        #Estimating total cost for given quantity
    return total
```



This function calculates the total cost of the manufacturing cycle, including material and process costs for this part.

15. Display_cost_breakdown

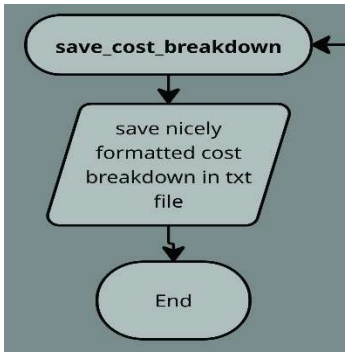
```python
def display_cost_breakdown(material_name, process_name, quantity,
                           mass_per_part_kg, material_cost_per_part,
                           process_cost_per_part, total_cost):
    print("\n=== COST BREAKDOWN FOR THE PRODUCTION OF THIS PART ===")        #Printing a comprehensive cost breakdown of the cost of making the certain number of parts for the user
    print(f"Material: {material_name}")
    print(f"Mass per part (kg): {mass_per_part_kg:.4f}")
    print(f"Material cost per part: $({material_cost_per_part:.2f}")
    print(f"Process: {process_name}")
    print(f"Process cost per part: $({process_cost_per_part:.2f}")
    print(f"Quantity: {quantity}")
    print("----------------------------------------------------")
    print(f"Estimated total production cost: $({total_cost:.2f})")
```



This function displays the cost breakdown for the user to see.

16. Save_cost_breakdown

```python
def save_cost_breakdown(material_name, process_name, quantity,
                        mass_per_part_kg, material_cost_per_part,        #creating a beautifully formatted cost breakdown report to be written to a text file
                        process_cost_per_part, total_cost, out_path):
    report = f"""
----------------------------------------
        COST BREAKDOWN REPORT
----------------------------------------

PROCESS SELECTED:  {process_name}
MATERIAL SELECTED: {material_name}
QUANTITY:          {quantity} parts

----------------------------------------
        MATERIAL COST ANALYSIS
----------------------------------------
- Unit Material Cost:       $({material_cost_per_part:.2f} per kg
- Part Mass:                {mass_per_part_kg:.3f} kg
- Material Required:        {mass_per_part_kg*quantity:.3f} kg
- Material Subtotal:        $({quantity*material_cost_per_part:.2f}

----------------------------------------
        PROCESS COST ANALYSIS
----------------------------------------
- Process Cost per Part:    $({process_cost_per_part:.2f} per part
- Processing Subtotal:      $({quantity*process_cost_per_part:.2f}

----------------------------------------
            TOTAL COST
----------------------------------------
- TOTAL COST:               $({total_cost:.2f}
----------------------------------------
    """

    with open(out_path, "w") as f:                        #Saving the cost breakdown to a text file at the specified output path
        f.write(report)
```



This function saves the cost breakdown report to the outputs folder for future reference by the user.

4. References

1. Scikit-Learn. “Sklearn.ensemble.RandomForestClassifier — Scikit-Learn 0.20.3 Documentation.” *Scikit-Learn.org*, 2025, scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html.
2. scikit-learn. “Sklearn.tree.DecisionTreeRegressor — Scikit-Learn 0.23.2 Documentation.” *Scikit-Learn.org*, scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeRegressor.html.
3. python.org. “8. Errors and Exceptions — Python 3.8.1 Documentation.” *Python.org*, 2020, docs.python.org/3/tutorial/errors.html.
4. Python Software Foundation. “Os — Miscellaneous Operating System Interfaces — Python 3.8.0 Documentation.” *Python.org*, docs.python.org/3/library/os.html.

5. Appendix

***1.* User manual**

- **Step 1 — Enter material constraints (user inputs)**

When prompted, type the prepared input material values and press Enter for each.

Example interaction:

```
PS C:\Users\cnvik\OneDrive\Purdue\ENGR_133\Individual Project\Manufacturing Made Easier> & C:/Users/cnvik/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/cnvik/OneDr
ive/Purdue/ENGR_133/Individual Project/Manufacturing Made Easier/main.py"
====== Manufacturing Made Easier (Material, Process, and Cost Estimator) ======
Enter minimum required yield strength (MPa): 200
Enter max allowable density (g/cm^3): 3.0
Enter maximum cost ($/kg): 15
```

- **Step 2 — Top material recommendations**

The program trains/loads the material model, computes combined ML+heuristic scores, and prints the top recommended materials (usually top 3).

Example output:

```
Material model trained.
R2=0.855(Higher = Better),
MAE=4.681(Lower = Better)

Searching for materials that meet your constraints...
Constraints will be relaxed by 10% per round of relaxing the constraints, if no materials meet your constraints.

Top recommended materials:
1. Aluminum 5083 | Score: 79.0 | Cost: $3.2/kg | Density: 2.66 g/cm3
2. Aluminum 2024 | Score: 78.6 | Cost: $4.8/kg | Density: 2.78 g/cm3
3. Aluminum 6061 | Score: 78.4 | Cost: $3.5/kg | Density: 2.7 g/cm3
```

- **Step 3 — Select a material**

The program will prompt:
Select material to use (1, 2 or 3):
Type the index number of your choice (e.g., 1) and press Enter. The program will echo the selected material and its properties.

```
Select material to use (1, 2 or 3): 1

Selected material:
name                          Aluminum 5083
density_g_cm3                         2.66
yield_strength_MPa                     290
cost_usd_per_kg                        3.2
corrosion_rating                       4.0
predicted_score                  78.956336
quantitative_analysis_score           85.5
qualitative_analysis_score       63.687787
```

- **Step 4 — Enter part specifications**

Next, enter the part geometry and production quantity when prompted:

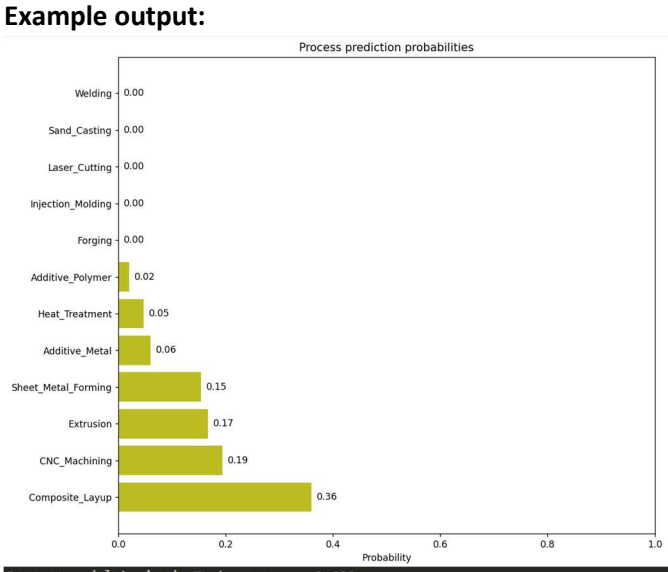
Example interaction:

```
Enter Part volume (cm^3): 120

Enter Surface area (cm^2): 350

Enter Tolerance (mm): 0.1

Enter Quantity (integer): 10
```

- **Step 5 — Process prediction and plot display**

The process model takes the selected material properties + part specs and prints the predicted process and confidence. It also displays the plot of process probabilities.

Example output:




```
Process model trained. Test accuracy: 0.600

Predicted process: Composite_Layup (confidence 0.360)
```

- **Step 6 — Cost calculation & console summary**

The program computes part mass, material cost per part, process cost per part (from PROCESS_COST_MAP), and total cost for the requested quantity. It prints a concise cost summary to terminal.

Example output:

```
=== COST BREAKDOWN FOR THE PRODUCTION OF THIS PART ===
Material: Aluminum 5083
Mass per part (kg): 0.3192
Material cost per part: $1.02
Process: Composite_Layup
Process cost per part: $30.00
Quantity: 10
--------------------------------------------------
Estimated total production cost: $310.21
```

- **Step 7 — Formatted cost report and Process probability plot saved to /output**

A neatly formatted .txt cost report and the process probability plot generated earlier is saved to the output/ folder. The program prints a success message and a thank you message to end.

Example Text File and Terminal Output:

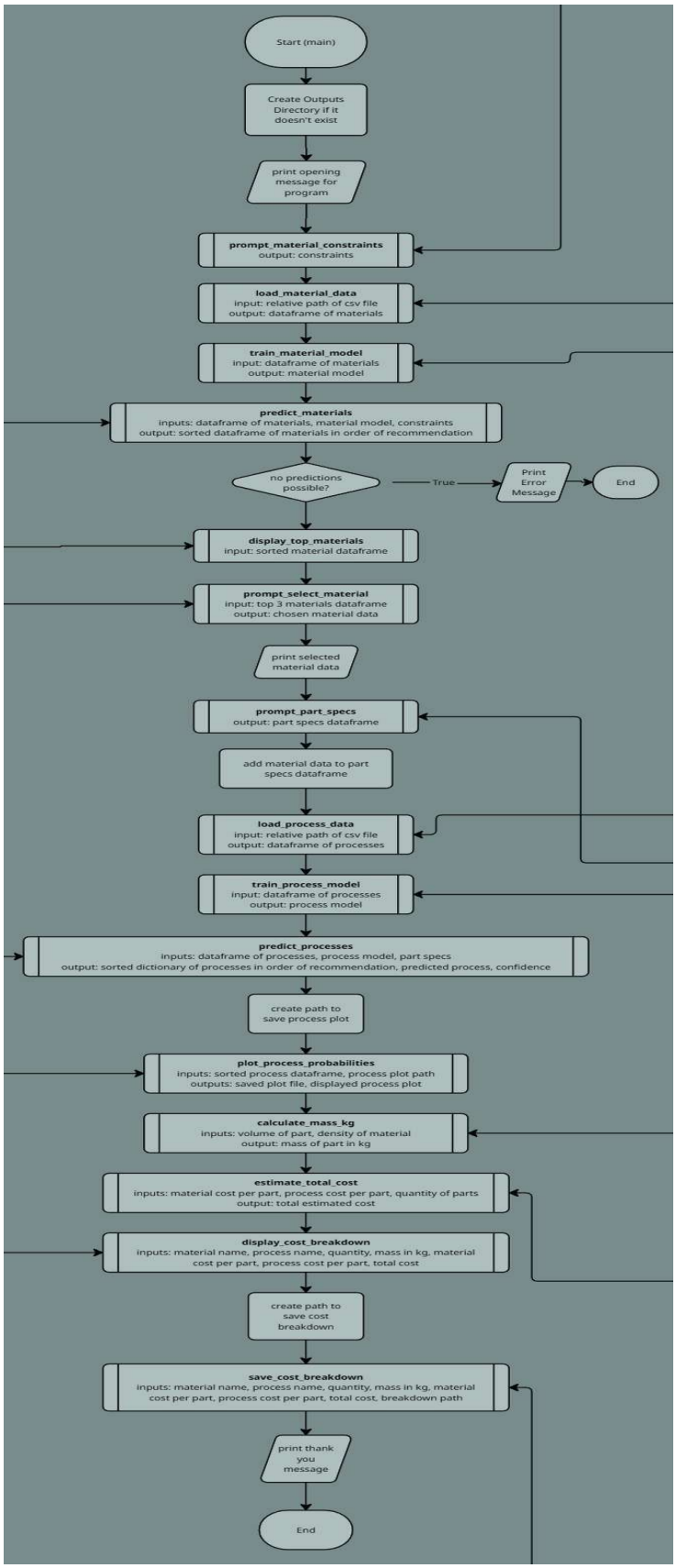
```
========================================
        COST BREAKDOWN REPORT
========================================

PROCESS SELECTED:  Composite_Layup
MATERIAL SELECTED: Aluminum 5083
QUANTITY:          10 parts

========================================
        MATERIAL COST ANALYSIS
========================================
- Unit Material Cost:       $1.02 per kg
- Part Mass:                0.319 kg
- Material Required:        3.192 kg
- Material Subtotal:        $10.21

========================================
        PROCESS COST ANALYSIS
========================================
- Process Cost per Part:    $30.00 per part
- Processing Subtotal:      $300.00

========================================
              TOTAL COST
========================================
- TOTAL COST:               $310.21
========================================
Process probability chart and Cost Breakdown Report saved to output folder.
All done. Thank you for using Manufacturing Made Easier!
```

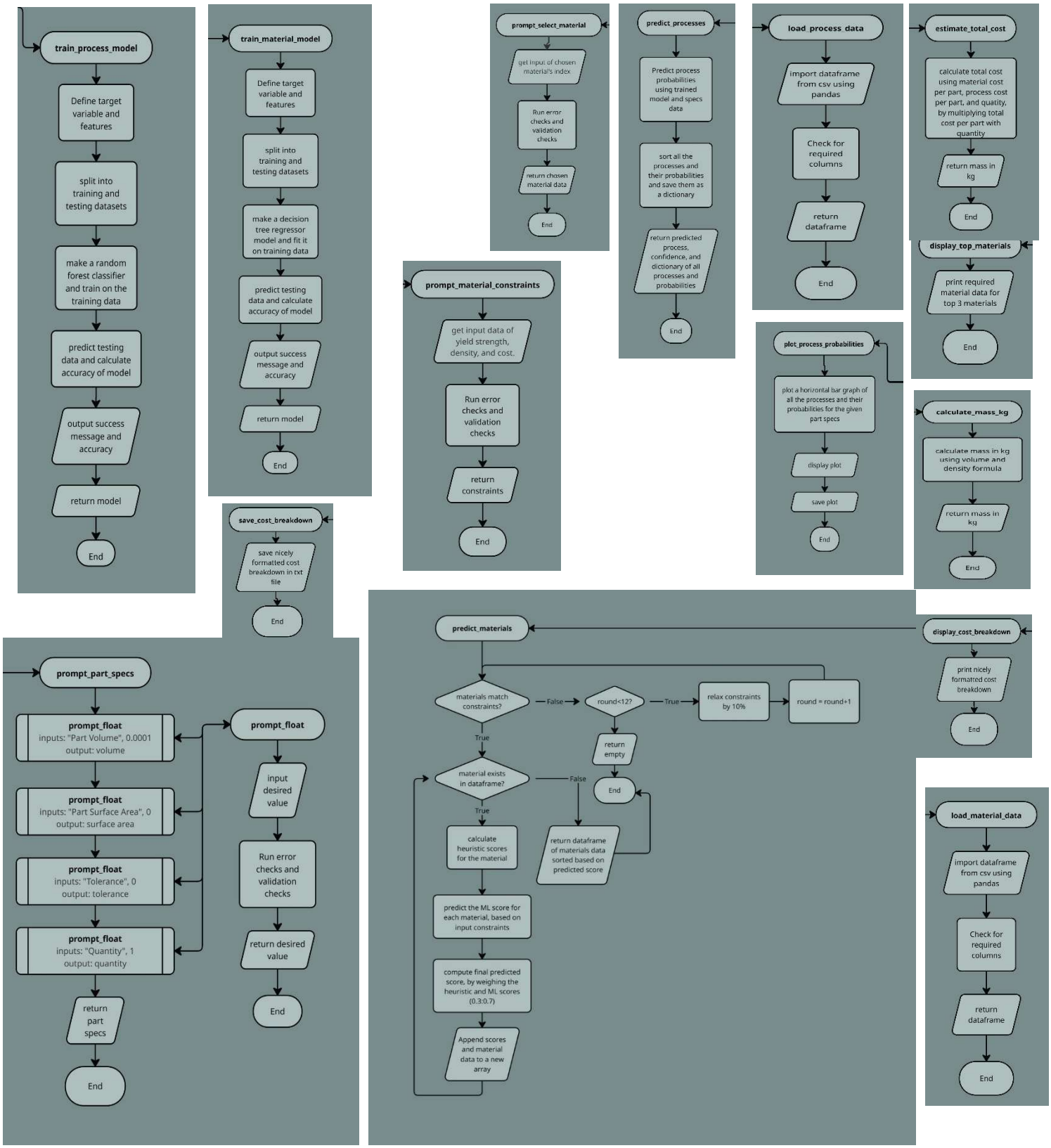
- **Additional Feature — Auto-relaxation behavior (if no materials match)**

If your initial material constraints are too strict and no materials satisfy them, the program will automatically relax constraints in small steps (reduce required strength by 10% per round, increase density by 10%, increase cost by 10%) until at least one material matches. The program prints each relaxation pass and the final relaxed values, or it ends if no materials still math the relaxed constraints.

```
PS C:\Users\cnvik\OneDrive\Purdue\ENGR_133\Individual Project\Manufacturing Made Easier> & C:/Users/cnvik/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/cnvik/OneDr
ive/Purdue/ENGR_133/Individual Project/Manufacturing Made Easier/main.py"
====== Manufacturing Made Easier (Material, Process, and Cost Estimator) ======
Enter minimum required yield strength (MPa): 1000000
Enter max allowable density (g/cm^3): 1212212
Enter maximum cost ($/kg): 0.0001
Material model trained.
R2=0.855(Higher = Better),
MAE=4.681(Lower = Better)

Searching for materials that meet your constraints...
Constraints will be relaxed by 10% per round of relaxing the constraints, if no materials meet your constraints.
Relaxing Constraints -- Round 1...
Relaxing Constraints -- Round 2...
Relaxing Constraints -- Round 3...
Relaxing Constraints -- Round 4...
Relaxing Constraints -- Round 5...
Relaxing Constraints -- Round 6...
Relaxing Constraints -- Round 7...
Relaxing Constraints -- Round 8...
Relaxing Constraints -- Round 9...
Relaxing Constraints -- Round 10...
Relaxing Constraints -- Round 11...
Relaxing Constraints -- Round 12...
Relaxing Constraints -- Round 13...

No materials found even after relaxation. Please try again with different constraints.
```

2. Flowchart


Start (main)
Create Outputs Directory if it doesn't exist
print opening message for program
prompt_material_constraints
output: constraints
load_material_data
input: relative path of csv file
output: dataframe of materials
train_material_model
input: dataframe of materials
output: material model
predict_materials
inputs: dataframe of materials, material model, constraints
output: sorted dataframe of materials in order of recommendation
no predictions possible?
True
Print Error Message
End
display_top_materials
input: sorted material dataframe
prompt_select_material
input: top 3 materials dataframe
output: chosen material data
print selected material data
prompt_part_specs
output: part specs dataframe
add material data to part specs dataframe
load_process_data
input: relative path of csv file
output: dataframe of processes
train_process_model
input: dataframe of processes
output: process model
predict_processes
inputs: dataframe of processes, process model, part specs
output: sorted dictionary of processes in order of recommendation, predicted process, confidence
create path to save process plot
plot_process_probabilities
inputs: sorted process dataframe, process plot path
outputs: saved plot file, displayed process plot
calculate_mass_kg
inputs: volume of part, density of material
output: mass of part in kg
estimate_total_cost
inputs: material cost per part, process cost per part, quantity of parts
output: total estimated cost
display_cost_breakdown
inputs: material name, process name, quantity, mass in kg, material cost per part, process cost per part, total cost
create path to save cost breakdown
save_cost_breakdown
inputs: material name, process name, quantity, mass in kg, material cost per part, process cost per part, total cost, breakdown path
print thank you message
End


train_process_model
Define target variable and features
split into training and testing datasets
make a random forest classifier and train on the training data
predict testing data and calculate accuracy of model
output success message and accuracy
return model
End
train_material_model
Define target variable and features
split into training and testing datasets
make a decision tree regressor model and fit it on training data
predict testing data and calculate accuracy of model
output success message and accuracy
return model
End
prompt_select_material
get input of chosen material's index
Run error checks and validation checks
return chosen material data
End
predict_processes
Predict process probabilities using trained model and specs data
sort all the processes and their probabilities and save them as a dictionary
return predicted process, confidence, and dictionary of all processes and probabilities
End
load_process_data
import dataframe from csv using pandas
Check for required columns
return dataframe
End
estimate_total_cost
calculate total cost using material cost per part, process cost per part, and quatity, by multiplying total cost per part with quantity
return mass in kg
End
display_top_materials
print required material data for top 3 materials
End
prompt_material_constraints
get input data of yield strength, density, and cost.
Run error checks and validation checks
return constraints
End
plot_process_probabilities
plot a horizontal bar graph of all the processes and their probabilities for the given part specs
display plot
save plot
End
calculate_mass_kg
calculate mass in kg using volume and density formula
return mass in kg
End
save_cost_breakdown
save nicely formatted cost breakdown in txt file
End
prompt_part_specs
prompt_float
inputs: "Part Volume", 0.0001
output: volume
prompt_float
inputs: "Part Surface Area", 0
output: surface area
prompt_float
inputs: "Tolerance", 0
output: tolerance
prompt_float
inputs: "Quantity", 1
output: quantity
return part specs
End
prompt_float
input desired value
Run error checks and validation checks
return desired value
End
predict_materials
materials match constraints?
False
round<12?
True
relax constraints by 10%
round = round+1
True
return empty
End
material exists in dataframe?
False
True
calculate heuristic scores for the material
return dataframe of materials data sorted based on predicted score
predict the ML score for each material, based on input constraints
compute final predicted score, by weighing the heuristic and ML scores (0.3:0.7)
Append scores and material data to a new array
display_cost_breakdown
print nicely formatted cost breakdown
End
load_material_data
import dataframe from csv using pandas
Check for required columns
return dataframe
End

***3.* Code**

Main.py:

```
"""
Course Number: ENGR 13300
Semester: Fall 2025

Description:
   This program ties together the material, process, and cost modules to provide
   a comprehensive manufacturing recommendation and cost estimation tool.

Assignment Information:
   Assignment:     Individual Project(Manufacturing Made Easier) - Main Module
   Team ID:        LC5-10
   Author:         Chaitanya Nikhil Vikamsey, cvikamse@purdue.edu
   Date:           11/30/2025

Contributors:
   None

   My contributor(s) helped me:
   [ ] understand the assignment expectations without
      telling me how they will approach it.
   [ ] understand different ways to think about a solution
      without helping me plan my solution.
   [ ] think through the meaning of a specific error or
      bug present in my code without looking at my code.
   Note that if you helped somebody else with their code, you
   have to list that person as a contributor here as well.

Academic Integrity Statement:
   I have not used source code obtained from any unauthorized
   source, either modified or unmodified; nor have I provided
   another student access to my code.  The project I am
   submitting is my own original work.
"""

import os

from material_module import (
   prompt_material_constraints,
```

```
    load_material_data,
    train_material_model,
    predict_materials,
    display_top_materials,
)
from process_module import (
    prompt_part_specs,
    load_process_data,
    train_process_model,
    predict_processes,
    plot_process_probabilities,
)
from cost_module import (
    calculate_mass_kg,
    estimate_total_cost,
    display_cost_breakdown,
    save_cost_breakdown,
    PROCESS_COST_MAP,
)



def prompt_select_material(top3_df):
    while True:                         #Prompting user to select one of the top 3 recommended materials, using try except
and an infinite loop until we get a valid value.
        try:
            choice = input("Select material to use (1, 2 or 3): ").strip()
            if choice not in ("1", "2", "3"):
                print("Please enter 1, 2, or 3.")
                continue
            id = int(choice) - 1
            selected = top3_df.iloc[id]
            return selected
        except Exception as e:
            print("Invalid choice:", e)



def main():
    OUTPUT_DIR = "output"
    os.makedirs(OUTPUT_DIR, exist_ok=True)              #Creating output directory if it doesn't exist, so that I can
store the process probability plots there
    print("====== Manufacturing Made Easier (Material, Process, and Cost Estimator) ======")
    #collecting user constraints for material selection
    constraints = prompt_material_constraints()
    #loading material data and training material selection model
    mat_df = load_material_data("data/materials.csv")
    mat_model = train_material_model(mat_df)
```

```
    #predicting materials based on user constraints and displaying the top recommendations
    mat_predictions = predict_materials(mat_model, mat_df, constraints)
    if mat_predictions.empty:
        print("\nNo materials found even after relaxation. Please try again with different constraints.")   #handling case
where no materials are found even after constraint relaxation
    else:
        display_top_materials(mat_predictions)
        #prompting user to select one of the top 3 materials and then printing the selected material and its properties
        top3 = mat_predictions.head(3)
        selected_material = prompt_select_material(top3)
        print("\nSelected material:")
        print(selected_material.to_string())
        #collecting part specifications from user and adding material properties to it
        part_specs = prompt_part_specs()
        part_specs["material_name"] = selected_material["name"]
        part_specs["material_density_g_cm3"] = float(selected_material["density_g_cm3"])
        part_specs["material_strength_MPa"] = float(selected_material["yield_strength_MPa"])
        part_specs["material_cost_usd_per_kg"] = float(selected_material["cost_usd_per_kg"])
        #loading process data, training process model, predicting processes for the part, and plotting the process
probabilities
        proc_df = load_process_data("data/process_train.csv")
        proc_model = train_process_model(proc_df)
        proc_result = predict_processes(proc_model, part_specs)
        proc_plot_path = os.path.join(OUTPUT_DIR, "process_probabilities.png")
        plot_process_probabilities(proc_result, proc_plot_path)
        #calculating mass, costs, and displaying & saving cost breakdown
        mass_kg = calculate_mass_kg(part_specs["volume_cm3"], part_specs["material_density_g_cm3"])
        process_name = proc_result["predicted_process"]
        material_cost_per_part = part_specs["material_cost_usd_per_kg"] * mass_kg
        process_cost_per_part = PROCESS_COST_MAP.get(process_name, PROCESS_COST_MAP["Other"])
        total_cost = estimate_total_cost(material_cost_per_part, process_cost_per_part, part_specs["quantity"])
        display_cost_breakdown(
            selected_material["name"],
            process_name,
            part_specs["quantity"],
            mass_kg,
            material_cost_per_part,
            process_cost_per_part,
            total_cost
        )
        out_report_path = os.path.join(OUTPUT_DIR, "cost_breakdown_report.txt")
        save_cost_breakdown(
            selected_material["name"],
            process_name,
            part_specs["quantity"],
            mass_kg,
            material_cost_per_part,
            process_cost_per_part,
```

```
            total_cost,
            out_report_path
        )
        #ends with a thank you message
        print("\nProcess probability chart and Cost Breakdown Report saved to output folder.")
        print("All done. Thank you for using Manufacturing Made Easier!")

if __name__ == "__main__":
    main()
```

Material_module.py:

```
"""
Course Number: ENGR 13300
Semester: Fall 2025

Description:
   This program contains functions that are user defined to prompt the user for constraints,
   load material data, train a material recommendation model, predict suitable materials based on
   user constraints, and display the top recommended materials.

Assignment Information:
   Assignment:     Individual Project(Manufacturing Made Easier) - Material Module
   Team ID:        LC5-10
   Author:         Chaitanya Nikhil Vikamsey, cvikamse@purdue.edu
   Date:           11/30/2025

Contributors:
   None

   My contributor(s) helped me:
   [ ] understand the assignment expectations without
      telling me how they will approach it.
   [ ] understand different ways to think about a solution
      without helping me plan my solution.
   [ ] think through the meaning of a specific error or
      bug present in my code without looking at my code.
   Note that if you helped somebody else with their code, you
   have to list that person as a contributor here as well.

Academic Integrity Statement:
   I have not used source code obtained from any unauthorized
   source, either modified or unmodified; nor have I provided
   another student access to my code.  The project I am
   submitting is my own original work.
"""

import pandas as pd
```

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_absolute_error

def prompt_material_constraints():
    while True:
        try:                                        #User input with validation, using try except blocks
            strength = float(input("Enter minimum required yield strength (MPa): "))
            if strength <= 0:
                print("Strength must be greater than 0.")
                continue
            break
        except ValueError:
            print("Invalid input. Please enter a numeric value.")

    while True:
        try:
            density = float(input("Enter max allowable density (g/cm^3): "))
            if density <= 0:
                print("Density must be greater than 0.")
                continue
            break
        except ValueError:
            print("Invalid input. Please enter a numeric value.")

    while True:
        try:
            cost = float(input("Enter maximum cost ($/kg): "))
            if cost <= 0:
                print("Cost must be greater than 0.")
                continue
            break
        except ValueError:
            print("Invalid input. Please enter a numeric value.")

    return {                        #Returning constraints as a dictionary
        "target_strength": strength,
        "max_density": density,
        "max_cost": cost
    }


def load_material_data(path):
    df = pd.read_csv(path)
    required = ["name", "density_g_cm3", "yield_strength_MPa", "cost_usd_per_kg",
                "corrosion_rating", "success_score"]
    for col in required:                        #Checking for required columns in the CSV
        if col not in df.columns:
```

```
        raise ValueError(f"Missing column '{col}' in materials CSV.")
    return df.copy()                    #Returning a copy of the dataframe


def train_material_model(df):
    features = df[["density_g_cm3", "yield_strength_MPa", "cost_usd_per_kg",
                   "corrosion_rating"]].astype(float)                    #Defining features and target variable for model training
    target = df["success_score"].astype(float)

    X_train, X_test, y_train, y_test = train_test_split(features, target, test_size=0.2, random_state=42)      #splits data into
train and test
    model = DecisionTreeRegressor(max_depth=5, random_state=42)                #Creating a decision tree regressor
model
    model.fit(X_train, y_train)                    #Training the decision tree regressor model on training data

    preds = model.predict(X_test)                #Making predictions on test data
    metrics = {
        "r2": float(r2_score(y_test, preds)),
        "mae": float(mean_absolute_error(y_test, preds))
    }
    print(f"Material model trained. \nR2={metrics['r2']:.3f}(Higher = Better), \nMAE={metrics['mae']:.3f}(Lower =
Better)")     #Printing model performance metrics, for user to know accuracy of Model
    return model

def predict_materials(model, df, constraints):
    relaxation_round = 0
    print("\nSearching for materials that meet your constraints... \nConstraints will be relaxed by 10% per round of
relaxing the constraints, if no materials meet your constraints.")
    while True:
        #only materials meeting constraints remain
        df = df[
            (df["yield_strength_MPa"] >= constraints["target_strength"]) &
            (df["density_g_cm3"] <= constraints["max_density"]) &
            (df["cost_usd_per_kg"] <= constraints["max_cost"])
            ]

        #If at least one material matches, don't relax constraints
        if not df.empty:
            break

        #If no materials found, then relax constraints

        # asking the user to reconsider their constraints if too many relaxations have occurred
        if relaxation_round > 12:   # roughly 12 relax cycles, should be enough, and if it isnt, the constraints are too tight
            return pd.DataFrame([])  # return empty datafame to indicate no filtering possible, and constraints are too tight
        else:
            relaxation_round += 1
            print(f"Relaxing Constraints -- Round {relaxation_round}...")
```

```
        #Relaxation rules
        constraints["target_strength"] *= 0.9      # reduce strength by 10%
        constraints["max_density"] *= 1.1          # increase density by 10%
        constraints["max_cost"] *= 1.1             # increase cost by 10%

    if 12 > relaxation_round > 0:     # Inform user of final relaxed values, if it happens within 12 cycles
        print("\n Constraints relaxed to allow valid materials:")
        print(f"  Final target strength: {constraints["target_strength"]:.2f} MPa")
        print(f"  Final max density:     {constraints["max_density"]:.2f} g/cm³")
        print(f"  Final max cost:        ${constraints["max_cost"]:.2f}/kg")

    # Scoring remaining materials with the trained model
    rows = []
    for _, row in df.iterrows():
      # Calculating heuristic scores based on constraints for qualitative aspect of analysis and material recommendation
      # Strength score
      s_strength = min(row["yield_strength_MPa"] / max(1.0, constraints.get("target_strength", 1.0)), 1.0)
      # Cost score
      s_cost = 1 - min(row["cost_usd_per_kg"] / max(1.0, constraints.get("max_cost", row["cost_usd_per_kg"])), 1.0)
      # Density score
      dens_min = df["density_g_cm3"].min()
      dens_max = df["density_g_cm3"].max()
      if dens_max - dens_min > 0:
        s_density = 1 - (row["density_g_cm3"] - dens_min) / (dens_max - dens_min)
      else:
        s_density = 0.5

      # Predicting quantitative score using the trained decision tree regressor model
      X = pd.DataFrame([{
        "density_g_cm3": row["density_g_cm3"],
        "yield_strength_MPa": row["yield_strength_MPa"],
        "cost_usd_per_kg": row["cost_usd_per_kg"],
        "corrosion_rating": row["corrosion_rating"]
      }])
      predicted = model.predict(X)[0]

      # Combining quantitative and qualitative scores for final material recommendation
      heuristic = ((s_strength + s_cost + s_density) / 3.0)*100
      combined_score = 0.7 * predicted + 0.3 * (heuristic)

      rows.append({
        "name": row["name"],
        "density_g_cm3": row["density_g_cm3"],
        "yield_strength_MPa": row["yield_strength_MPa"],
        "cost_usd_per_kg": row["cost_usd_per_kg"],
        "corrosion_rating": row["corrosion_rating"],
        "predicted_score": combined_score,
        "quantitative_analysis_score": predicted,
```

```
        "qualitative_analysis_score": heuristic
    })

  out = pd.DataFrame(rows)
  out = out.sort_values("predicted_score", ascending=False).reset_index(drop=True) #Sorting materials by predicted score in descending order
  return out

def display_top_materials(df, n=3):
  print("\nTop recommended materials:")      #Displaying top N recommended materials, keeping n as 3 by default, but changeable as needed
  for i in range(min(n, len(df))):
    r = df.iloc[i]
    print(f"{i+1}. {r['name']} | Score: {r['predicted_score']:.1f} | Cost: ${r['cost_usd_per_kg']}/kg | Density: {r['density_g_cm3']} g/cm3")
```

Process_module.py:

```
"""
Course Number: ENGR 13300
Semester: Fall 2025

Description:
  This program contains functions that are user defined to load process data, train a process recommendation
  model (Random Forset Classifiers), prompt the user for part specifications, predict suitable manufacturing
  processes based on part specifications, and plot the process prediction probabilities.

Assignment Information:
  Assignment:    Individual Project(Manufacturing Made Easier) - Process Module
  Team ID:       LC5-10
  Author:        Chaitanya Nikhil Vikamsey, cvikamse@purdue.edu
  Date:          11/30/2025

Contributors:
  None

  My contributor(s) helped me:
  [ ] understand the assignment expectations without
    telling me how they will approach it.
  [ ] understand different ways to think about a solution
    without helping me plan my solution.
  [ ] think through the meaning of a specific error or
    bug present in my code without looking at my code.
  Note that if you helped somebody else with their code, you
  have to list that person as a contributor here as well.

Academic Integrity Statement:
```

```
   I have not used source code obtained from any unauthorized
   source, either modified or unmodified; nor have I provided
   another student access to my code.  The project I am
   submitting is my own original work.
"""

import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

def load_process_data(path):
   df = pd.read_csv(path)
   required = ["volume_cm3", "surface_area_cm2", "tolerance_mm",
            "material_strength", "material_density", "process_label"]
   for col in required:                                #Validating presence of required columns in CSV
      if col not in df.columns:
         raise ValueError(f"Missing column '{col}' in process CSV.")
   return df.copy()

def train_process_model(df):
   X = df[["volume_cm3", "surface_area_cm2", "tolerance_mm",
         "material_strength", "material_density"]].astype(float)       #defining feature matrix and target vector
   y = df["process_label"].astype(str)
   X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
   model = RandomForestClassifier(n_estimators=150, random_state=42)              #Using Random Forest Classifier with
150 trees for better accuracy
   model.fit(X_train, y_train)                         #Training the model, and then evaluating on test set
   preds = model.predict(X_test)
   acc = accuracy_score(y_test, preds)
   print(f"Process model trained. Test accuracy: {acc:.3f}")          #Printing model accuracy on test set for the user to
know
   return model

def prompt_part_specs():
   def prompt_float(name, min_val=None):              #using a nested function to prompt for float inputs with validation
so that it is easier
      while True:                          #to call as long as a valid input is given
         try:
            s = input(f"\nEnter {name}: ").strip()
            val = float(s)
            if min_val is not None and val < min_val:
               print(f"{name} must be >= {min_val}. Try again.")
               continue
            return val
         except ValueError:
            print("Please enter a numeric value.")
```

```
    volume = prompt_float("Part volume (cm^3)", min_val=0.0001)          #Prompting user for part specifications using
previoulsy defined UDF
    surface_area = prompt_float("Surface area (cm^2)", min_val=0.0)
    tolerance = prompt_float("Tolerance (mm)", min_val=0.0)
    qty = int(prompt_float("Quantity (integer)", min_val=1))
    return {
        "volume_cm3": volume,
        "surface_area_cm2": surface_area,
        "tolerance_mm": tolerance,
        "quantity": qty
    }

def predict_processes(model, specs):
    X = pd.DataFrame([{                                  #Creating a dataframe from user input specifications for prediction
        "volume_cm3": specs["volume_cm3"],
        "surface_area_cm2": specs["surface_area_cm2"],
        "tolerance_mm": specs["tolerance_mm"],
        "material_strength": specs["material_strength_MPa"],
        "material_density": specs["material_density_g_cm3"],
    }])
    proba = model.predict_proba(X)[0]                          #Predicting process probabilities using the trained model
    classes = model.classes_                                   #Getting the class labels
    pairs = list(zip(classes, proba))                          #Zipping class labels with their corresponding probabilities
    pairs_sorted = sorted(pairs, key=lambda x: x[1], reverse=True)     #Sorting the pairs based on probabilities in
descending order
    predicted_process, confidence = pairs_sorted[0]                    #Getting the process with highest probability and its
confidence
    all_probs = {name: float(prob) for name, prob in pairs_sorted}     #Creating a dictionary of all process probabilities
    print(f"\nPredicted process: {predicted_process} (confidence {confidence:.3f})")
    return {"predicted_process": predicted_process, "confidence": float(confidence), "all_probs": all_probs}

def plot_process_probabilities(proc_result, out_path):
    probs = proc_result["all_probs"]                   #Plotting the process prediction probabilities as a horizontal bar chart
    names = list(probs.keys())
    vals = [probs[n] for n in names]

    plt.figure(figsize=(10, 10))
    plt.barh(names, vals, color="C8")
    plt.xlabel("Probability")
    plt.title("Process prediction probabilities")
    plt.xlim(0, 1)
    for i, v in enumerate(vals):
        plt.text(v + 0.01, i, f"{v:.2f}", va="center")         #Adding text labels to each bar indicating the probability value
    plt.tight_layout()
    plt.savefig(out_path, dpi=150)                             #Saving the plot to the specified output path
    plt.show()
    plt.close()
```

Cost_module.py:

```
"""
Course Number: ENGR 13300
Semester: Fall 2025

Description:
   This program contains UDFs to calculate mass, estimate total cost, and display a cost breakdown
   for manufacturing a part based on selected material and process.

Assignment Information:
   Assignment:     Individual Project(Manufacturing Made Easier) - Cost Module
   Team ID:       LC5-10
   Author:        Chaitanya Nikhil Vikamsey, cvikamse@purdue.edu
   Date:          11/30/2025

Contributors:
   None

   My contributor(s) helped me:
   [ ] understand the assignment expectations without
      telling me how they will approach it.
   [ ] understand different ways to think about a solution
      without helping me plan my solution.
   [ ] think through the meaning of a specific error or
      bug present in my code without looking at my code.
   Note that if you helped somebody else with their code, you
   have to list that person as a contributor here as well.

Academic Integrity Statement:
   I have not used source code obtained from any unauthorized
   source, either modified or unmodified; nor have I provided
   another student access to my code.  The project I am
   submitting is my own original work.
"""

PROCESS_COST_MAP = {                    #Process cost per part in USD, a sample mapping based on typical industry costs
   "Sand_Casting": 7.0,
   "Forging": 18.0,
   "CNC_Machining": 22.0,
   "Sheet_Metal_Forming": 10.0,
   "Additive_Metal": 35.0,
   "Additive_Polymer": 12.0,
   "Welding": 8.0,
   "Injection_Molding": 5.0,
   "Extrusion": 6.0,
   "Laser_Cutting": 14.0,
   "Composite_Layup": 30.0,
```

```
    "Heat_Treatment": 9.0,
    "Other": 12.0
}


def calculate_mass_kg(volume_cm3, density_g_cm3):
    mass = (float(volume_cm3) * float(density_g_cm3)) / 1000.0        #Calculating mass in kg from volume in cm^3 and
density in g/cm^3
    return mass

def estimate_total_cost(material_cost_per_part, process_cost_per_part, quantity):
    total = (float(material_cost_per_part) + float(process_cost_per_part)) * int(quantity)        #Estimating total cost for
given quantity
    return total

def display_cost_breakdown(material_name, process_name, quantity,
                           mass_per_part_kg, material_cost_per_part,
                           process_cost_per_part, total_cost):
    print("\n=== COST BREAKDOWN FOR THE PRODUCTION OF THIS PART ===")        #Printing a comprehensive cost
breakdown of the cost of making the certain number of parts for the user
    print(f"Material: {material_name}")
    print(f"Mass per part (kg): {mass_per_part_kg:.4f}")
    print(f"Material cost per part: ${material_cost_per_part:.2f}")
    print(f"Process: {process_name}")
    print(f"Process cost per part: ${process_cost_per_part:.2f}")
    print(f"Quantity: {quantity}")
    print("----------------------------------------------------")
    print(f"Estimated total production cost: ${total_cost:.2f}")

def save_cost_breakdown(material_name, process_name, quantity,
                        mass_per_part_kg, material_cost_per_part,            #creating a beautifully formatted cost breakdown
report to be written to a text file
                        process_cost_per_part, total_cost, out_path):
    report = f"""
========================================
      COST BREAKDOWN REPORT
========================================

PROCESS SELECTED:  {process_name}
MATERIAL SELECTED: {material_name}
QUANTITY:          {quantity} parts

========================================
      MATERIAL COST ANALYSIS
========================================
- Unit Material Cost:      ${material_cost_per_part:.2f} per kg
- Part Mass:               {mass_per_part_kg:.3f} kg
- Material Required:       {mass_per_part_kg*quantity:.3f} kg
```

```
- Material Subtotal:       ${quantity*material_cost_per_part:.2f}

========================================
      PROCESS COST ANALYSIS
========================================
- Process Cost per Part:    ${process_cost_per_part:.2f} per part
- Processing Subtotal:      ${quantity*process_cost_per_part:.2f}

========================================
           TOTAL COST
========================================
- TOTAL COST:              ${total_cost:.2f}
========================================
   """

   with open(out_path, "w") as f:                    #Saving the cost breakdown to a text file at the specified output path
      f.write(report)
```